

About this Continuous Assessment

Assessment	11
Delivery method	Canvas submission of this file after completion
Deadline	As specified on Canvas.
ILOs	Be able to apply hybrid force/motion controller in a contact-rich task.

Allowed packages

Do not modify these. Do not add or remove packages.

```
In [ ]: # Helper functions and packages
%pip install numpy
%pip install matplotlib
```

Allowed imports

Do not modify, add, or remove imports. Do not add imports anywhere else in this notebook.

```
In [ ]: # Helper functions and packages
import numpy as np
```

Instructions

This assessment is made of 2 tasks. There are two points in total for this part of the assessment.

Each cell is tagged for the assessment, so do not delete or otherwise tamper with the cells. The assignment is to add content to each of the Python cells as requested.

Verify the spreadsheet

Your student ID will be used to assign you parameters. For each task, look into your row to see what you must implement.

Control of a 7-DoF Manipulator for a contact-rich tooling task

The experimental platform consists of a 7-DoF robotic manipulator equipped with an end-effector tool for contact-rich interaction task (sliding) on a planar surface.

A coordinate frame $o_c x_c y_c z_c$ is defined as the constraint frame, which is attached to the task space as shown in the figure.

Assuming that the contact between the tool and the surface is rigid and frictionless.



Figure 1 - A 7-DoF Kinova manipulator to conduct a fissure detection task on a planar surface.

Task 1 - Define the Task-Space Selection Matrices

The coordinates in constraint frame are defined as:

$$\mathbf{X} = [x \quad y \quad z \quad \theta_x \quad \theta_y \quad \theta_z]^T$$

where:

- x, y : translational motion along the surface plane,
- z : translational motion normal to the surface,
- $\theta_x, \theta_y, \theta_z$: rotational motion of the end-effector.

Complete the following code to define:

- S_f : the selection matrix for the constraint (force-controlled) subspace,
- S_v : the selection matrix for the admissible motion subspace.

The matrices should correctly separate the task-space directions associated with motion control and force control.

```
In [ ]: # Code for Task 1
# Selection matrix for the constraint subspace
Sf = np.diag([None, None, None, None, None, None])

# Selection matrix for the admissible motion subspace
Sv = np.diag([None, None, None, None, None, None])
```

```
In [ ]: # Marking variables (do not modify)
ansT1_Sf = Sf
ansT1_Sv = Sv
# End of code for Task 1
```

Task 2 - Compute the Joint-Space Force Control Input

In this task, you will implement the force-control component of the hybrid force/motion controller described in the lecture notes, using a proportional feedback controller with feedforward force compensation.

You are given the desired force F_d , measured force F_e , force-control gain K_f , Jacobian matrix J . Use the constraint selection matrix S_f you defined in Task 1.

Complete the code below to compute the joint-space force-control input u_f .

The implemented controller should include:

- a feedforward term to apply the desired contact force,
- and a feedback term to compensate for force tracking error.

```
In [ ]: # Define Force Control Parameters for Task 2
#####
# TODO:
# Enter the force control parameters below (Find them in the spreadsheet)
Kf = None           # Proportional gain for force feedback control
Fd_z = None         # Desired contact force in the zc direction (normal)
Fe_x = None         # Measured contact force in the xc direction
Fe_y = None         # Measured contact force in the yc direction
Fe_z = None         # Measured contact force in the zc direction
Fe_tau_x = None     # Measured contact torque in the xc direction
Fe_tau_y = None     # Measured contact torque in the yc direction
Fe_tau_z = None     # Measured contact torque in the zc direction
#####
```

```
In [ ]: # Code for Task 2
# Given values
Fd = np.array([0.0, 0.0, Fd_z, 0.0, 0.0, 0.0]) # Desired task-space force/wren
Fe = np.array([Fe_x, Fe_y, Fe_z, Fe_tau_x, Fe_tau_y, Fe_tau_z]) # Measured tas

# Jacobian for the current configuration
```

```
J = np.array([
    [ 0.10,  0.20,  0.05,  0.00,  0.00,  0.00,  0.00],
    [ 0.00,  0.10,  0.20,  0.10,  0.00,  0.00,  0.00],
    [ 0.30,  0.10,  0.00,  0.20,  0.10,  0.00,  0.00],
    [ 0.00,  0.00,  0.10,  0.20,  0.30,  0.10,  0.00],
    [ 0.00,  0.10,  0.00,  0.00,  0.20,  0.30,  0.10],
    [ 0.10,  0.00,  0.00,  0.10,  0.00,  0.20,  0.30]
])

# TODO: compute joint-space control input for the force control
uf = None

print("Joint-space force control input:")
print(uf)
```

```
In [ ]: # Marking variables (don't modify)
ansT2 = uf
# End of code for Task 2
```